



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

3. LAB 1: Introduzione ai passi LLVM

Compilatori – Middle end [I215-014]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-215]
Anno accademico 2025/2026

Prof. Andrea Marongiu
andrea.marongiu@unimore.it

Copyright note

È vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

È inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia.

Credits

- Cooper, Torczon, “Engineering a Compiler”, Elsevier
- Aho, Lam, Sethi, Ullman, “Compilatori: principi, tecniche e strumenti – seconda edizione”, Pearson
- Gibbons, Carnegie Mellon University, “Optimizing Compilers”
- Pekhimenko, University of Toronto, “Compiler Optimization”



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

INSTALLAZIONE LLVM 19

Disclaimer

Ma perché LLVM 19?

Siamo già alla 22...

LLVM cambia continuamente piccoli dettagli nelle API, soprattutto nella parte:

- PassManager
- registrazione dei plugin
- analysis manager
- pipeline parsing

Anche micro-cambiamenti possono farci perdere un sacco di tempo

Potete ovviamente installare la versione che preferite, ma non potremo dedicare tempo in aula a risolvere problemi legati a versioni diverse.

Installazione LLVM 19.1.7

Opzione 1) APT – Dai repository ufficiali

<https://apt.llvm.org/>

Cercate il vostro OS a questo link

Installazione LLVM 19.1.7

Opzione 2) Binaries – Dai repository ufficiali

<https://github.com/llvm/llvm-project/releases/tag/llvmorg-19.1.7>

 lldb-19.1.7.src.tar.xz	10.2 MB	Jan 14
 lldb-19.1.7.src.tar.xz.sig	438 Bytes	Jan 14
 LLVM-19.1.7-Linux-X64.tar.xz	1.54 GB	Jan 15
 LLVM-19.1.7-Linux-X64.tar.xz.jsonl	10.2 KB	Jan 15
 LLVM-19.1.7-macOS-ARM64.tar.xz	1.32 GB	Jan 15
 LLVM-19.1.7-macOS-ARM64.tar.xz.jsonl	10.1 KB	Jan 15
 LLVM-19.1.7-macOS-X64.tar.xz	1.47 GB	Jan 15
 LLVM-19.1.7-macOS-X64.tar.xz.jsonl	10.2 KB	Jan 15
 LLVM-19.1.7-win32.exe	311 MB	Jan 15
 LLVM-19.1.7-win32.exe.sig	543 Bytes	Jan 15

Installazione LLVM 19.1.7

Opzione 2) Binaries – Dai repository ufficiali

<https://github.com/llvm/llvm-project/releases/tag/llvmorg-19.1.7>

lldb-19.1.7.src.tar.xz

lldb-19.1.7.src.tar.xz.sig

LLVM-19.1.7-Linux-X64.tar.xz

LLVM-19.1.7-Linux-X64.tar.xz.jsonl

LLVM-19.1.7-macOS-ARM64.tar.xz

LLVM-19.1.7-macOS-ARM64.tar.xz.jsonl

LLVM-19.1.7-macOS-X64.tar.xz

LLVM-19.1.7-macOS-X64.tar.xz.jsonl

LLVM-19.1.7-win32.exe

LLVM-19.1.7-win32.exe.sig

Questa opzione vi permette di scaricare in locale una cartella con l'installazione completa con sottocartelle /bin, /include, /lib, etc. Potete puntare la vostra variabile d'ambiente \$PATH a questa cartella

```
> export PATH=<path/to/folder>/bin:$PATH
```

per puntare a questa installazione. Ottenete:

```
> which clang
```

```
> <path/to/folder>/bin/clang
```


Installazione LLVM 19.1.7

Opzione 3) Sorgente – Dai repository ufficiali

<https://github.com/llvm/llvm-project/releases/tag/llvmorg-19.1.7>

 polly-19.1.7.src.tar.xz.sig	438 Bytes	Jan 14
 runtimes-19.1.7.src.tar.xz	7.05 KB	Jan 14
 runtimes-19.1.7.src.tar.xz.sig	438 Bytes	Jan 14
 sources.jsonl	200 KB	Jan 14
 test-suite-19.1.7.src.tar.xz	163 MB	Jan 14
 test-suite-19.1.7.src.tar.xz.sig	438 Bytes	Jan 14
 third-party-19.1.7.src.tar.xz	444 KB	Jan 14
 third-party-19.1.7.src.tar.xz.sig	438 Bytes	Jan 14
 Source code (zip)		Jan 14
 Source code (tar.gz)		Jan 14

In fondo alla pagina



Installazione LLVM 19.1.7

Opzione 3) Sorgente – Dai repository ufficiali

<https://github.com/llvm/llvm-project/releases/tag/llvmorg-19.1.7>

 [polly-19.1.7.src.tar.xz.sig](#)
 [runtimes-19.1.7.src.tar.xz](#)
 [runtimes-19.1.7.src.tar.xz.sig](#)
 [sources.jsonl](#)
 [test-suite-19.1.7.src.tar.xz](#)
 [test-suite-19.1.7.src.tar.xz.sig](#)
 [third-party-19.1.7.src.tar.xz](#)
 [third-party-19.1.7.src.tar.xz.sig](#)

In fondo



 [Source code \(zip\)](#)
 [Source code \(tar.gz\)](#)

Segue nelle prossime slides la procedura per installare LLVM dai sorgenti.

Questa optione NON è quella consigliata, a meno che non vogliate avere una build locale

Installazione LLVM 19.1.7

- Creare nella vostra ROOT directory la seguente struttura

```
amarongiu@kernighan:~/workspace/LLVM_17$ ll
total 28
drwxr-xr-x  6 amarongiu compilers 4096 Mar  6 18:04 ./
drwxr-x---  6 amarongiu compilers 4096 Mar  3 12:17 ../
drwxr-xr-x 19 amarongiu compilers 4096 Mar  7 15:59 BUILD/
drwxr-xr-x  7 amarongiu compilers 4096 Mar  4 12:49 INSTALL/
-rw-r--r--  1 amarongiu compilers  88 Mar  6 18:04 setup.sh
drwxr-xr-x  3 amarongiu compilers 4096 Mar  3 12:27 SRC/
drwxr-xr-x  3 amarongiu compilers 4096 Mar  6 18:00 TEST/
```

Installazione LLVM 19.1.7

- Il file zip/tar appena scaricato va copiato in **SRC**, e lì scompattato

```
amarongiu@kernighan:~/workspace/LLVM_17$ ll
total 28
drwxr-xr-x  6 amarongiu compilers 4096 Mar  6 18:04 ./
drwxr-x---  6 amarongiu compilers 4096 Mar  3 12:17 ../
drwxr-xr-x 19 amarongiu compilers 4096 Mar  7 15:59 BUILD/
drwxr-xr-x  7 amarongiu compilers 4096 Mar  4 12:49 INSTALL/
-rw-r--r--  1 amarongiu compilers  88 Mar  6 18:04 setup.sh
drwxr-xr-x  3 amarongiu compilers 4096 Mar  3 12:27 SRC/
drwxr-xr-x  3 amarongiu compilers 4096 Mar  6 18:00 TEST/
```



Installazione LLVM 19.1.7

- La cartella **BUILD** conterrà il prodotto della compilazione dei sorgenti

```
amarongiu@kernighan:~/workspace/LLVM_17$ ll
total 28
drwxr-xr-x  6 amarongiu compilers 4096 Mar  6 18:04 ./
drwxr-x---  6 amarongiu compilers 4096 Mar  3 12:17 ../
drwxr-xr-x 19 amarongiu compilers 4096 Mar  7 15:59 BUILD/
drwxr-xr-x  7 amarongiu compilers 4096 Mar  4 12:49 INSTALL/
-rw-r--r--  1 amarongiu compilers  88 Mar  6 18:04 setup.sh
drwxr-xr-x  3 amarongiu compilers 4096 Mar  3 12:27 SRC/
drwxr-xr-x  3 amarongiu compilers 4096 Mar  6 18:00 TEST/
```



Installazione LLVM 19.1.7

- La cartella **INSTALL** conterrà i binari dei vari *tools*
 - clang, opt, lld, ...
- E le librerie, gli include files, etc.

```
amarongiu@kernighan:~/workspace/LLVM_17$ ll
total 28
drwxr-xr-x  6 amarongiu compilers 4096 Mar  6 18:04 ./
drwxr-x---  6 amarongiu compilers 4096 Mar  3 12:17 ../
drwxr-xr-x 19 amarongiu compilers 4096 Mar  7 15:59 BUILD/
drwxr-xr-x  7 amarongiu compilers 4096 Mar  4 12:49 INSTALL/
-rw-r--r--  1 amarongiu compilers   88 Mar  6 18:04 setup.sh
drwxr-xr-x  3 amarongiu compilers 4096 Mar  3 12:27 SRC/
drwxr-xr-x  3 amarongiu compilers 4096 Mar  6 18:00 TEST/
```



Installazione LLVM 19.1.7

- La cartella **TEST** conterrà le nostre esercitazioni
 - Ovvero i programmi che saranno oggetto delle nostre ottimizzazioni/analisi

```
amarongiu@kernighan:~/workspace/LLVM_17$ ll
total 28
drwxr-xr-x  6 amarongiu compilers 4096 Mar  6 18:04 ./
drwxr-x---  6 amarongiu compilers 4096 Mar  3 12:17 ../
drwxr-xr-x 19 amarongiu compilers 4096 Mar  7 15:59 BUILD/
drwxr-xr-x  7 amarongiu compilers 4096 Mar  4 12:49 INSTALL/
-rw-r--r--  1 amarongiu compilers  88 Mar  6 18:04 setup.sh
drwxr-xr-x  3 amarongiu compilers 4096 Mar  3 12:27 SRC/
drwxr-xr-x  3 amarongiu compilers 4096 Mar  6 18:00 TEST/
```



Installazione LLVM 19.1.7

- Potete crearvi un semplice *bash* script (`setup.sh`) per esportare la cartella dei tools `INSTALL/bin`
 - `export PATH=$ROOT/INSTALL/bin:$PATH`

```
amarongiu@kernighan:~/workspace/LLVM_17$ ll
total 28
drwxr-xr-x  6 amarongiu compilers 4096 Mar  6 18:04 ./
drwxr-x---  6 amarongiu compilers 4096 Mar  3 12:17 ../
drwxr-xr-x 19 amarongiu compilers 4096 Mar  7 15:59 BUILD/
drwxr-xr-x  7 amarongiu compilers 4096 Mar  4 12:49 INSTALL/
-rw-r--r--  1 amarongiu compilers   88 Mar  6 18:04 setup.sh
drwxr-xr-x  3 amarongiu compilers 4096 Mar  3 12:27 SRC/
drwxr-xr-x  3 amarongiu compilers 4096 Mar  6 18:00 TEST/
```



- Così facendo la vostra installazione può convivere con altre versioni di LLVM installate nel sistema

Installazione LLVM 19.1.7

- Potete crearvi un semplice *bash* script (`setup.sh`)

per eseguire l'installazione in una directory specifica.

- esempio:

```
$ which opt
ROOT/INSTALL/bin/opt
$
```

```
amarongiu@ubuntu:~$ ls -ld /usr/bin/opt
total 28
drwxr-xr-x 2 root root 4096 Nov 19 10:10
drwxr-xr-x 2 root root 4096 Nov 19 10:10
drwxr-xr-x 2 root root 4096 Nov 19 10:10
drwxr-xr-x 2 root root 4096 Nov 19 10:10
-rw-r--r-- 1 root root  120 Nov 19 10:10
drwxr-xr-x 2 root root 4096 Nov 19 10:10
drwxr-xr-x 2 root root 4096 Nov 19 10:10
```

- Così facendo la vostra installazione può convivere con altre versioni di LLVM installate nel sistema

Installazione LLVM 19.1.7

- L'installazione della toolchain prevede tre steps:
 - Configurazione
 - Compilazione
 - Installazione
- Possiamo fare riferimento alla documentazione ufficiale
 - <https://llvm.org/docs/GettingStarted.html#getting-started-with-llvm>

Configurazione

- `cd ROOT/BUILD`
- `cmake -G "Unix Makefiles"`
 - `-DCMAKE_BUILD_TYPE=Release`
 - `-DCMAKE_INSTALL_PREFIX=$ROOT/INSTALL`
 - `-DLLVM_ENABLE_PROJECTS="clang"`
 - `-DLLVM_TARGETS_TO_BUILD=host`
 - [other options]*`$ROOT/SRC/llvm-project-llvmorg-17.0.6/llvm/`

Compilazione

- `cd ROOT/BUILD` (dovreste già trovarvi qui)
- `make -j2` (l'opzione indica l'utilizzo di più cores per la compilazione)

NOTA: L'opzione `-j` comporta l'incremento dell'utilizzo della RAM. Verificare che il numero di cores indicato non comporti un sovraccarico della stessa ed eventualmente ridurlo (o disattivare completamente l'opzione)

Installazione

- `cd ROOT/BUILD` (dovreste già trovarvi qui)
- `make install`
- Al termine del processo tutti i tools si troveranno installati in `$ROOT/INSTALL`



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

Scrivere un passo LLVM

Ricorda: I *passi* LLVM

- Abbiamo visto che il middle-end è organizzato come una sequenza di *passi*
 - *Passi* di analisi
 - Consumano la IR e raccolgono informazioni sul programma
 - *Passi* di trasformazione
 - Trasformano il programma e producono nuova IR
- Perché esiste questo isolamento?
 - Migliore leggibilità
 - Diversi passi potrebbero richiedere la stessa informazione
 - L'isolamento evita analisi ridondante
- Per analizzare il codice e trasformarlo occorre una IR espressiva che mantenga tutte le informazioni importanti da trasmettere da un *passo* all'altro

La IR di LLVM

- La IR di LLVM ha una sintassi ed una semantica simile a quelle del linguaggio Assembly a cui siete abituati (es., RISC-V)

```
int main()  
{  
    return 0;  
}
```

```
define i32 @main() ...  
{  
    ret i32 0  
}
```


Concetti base di un passo LLVM

- Per rispondere a questa domanda bisogna prima comprendere i seguenti punti:
 - **Moduli LLVM:**
 - Come è tradotto il nostro programma in LLVM?
 - **Iteratori:**
 - Come attraversare il modulo?
 - **Downcasting:**
 - Come ricavare maggiori informazioni dagli iteratori?
 - **Interfacce dei passi LLVM:**
 - Che interfacce fornisce LLVM per scrivere i passi?

<https://llvm.org/docs/WritingAnLLVMNewPMPass.html>

<https://llvm.org/docs/WritingAnLLVMPass.html>

Legacy Pass Manager, sta gradualmente sparendo dal middle-end

Moduli LLVM

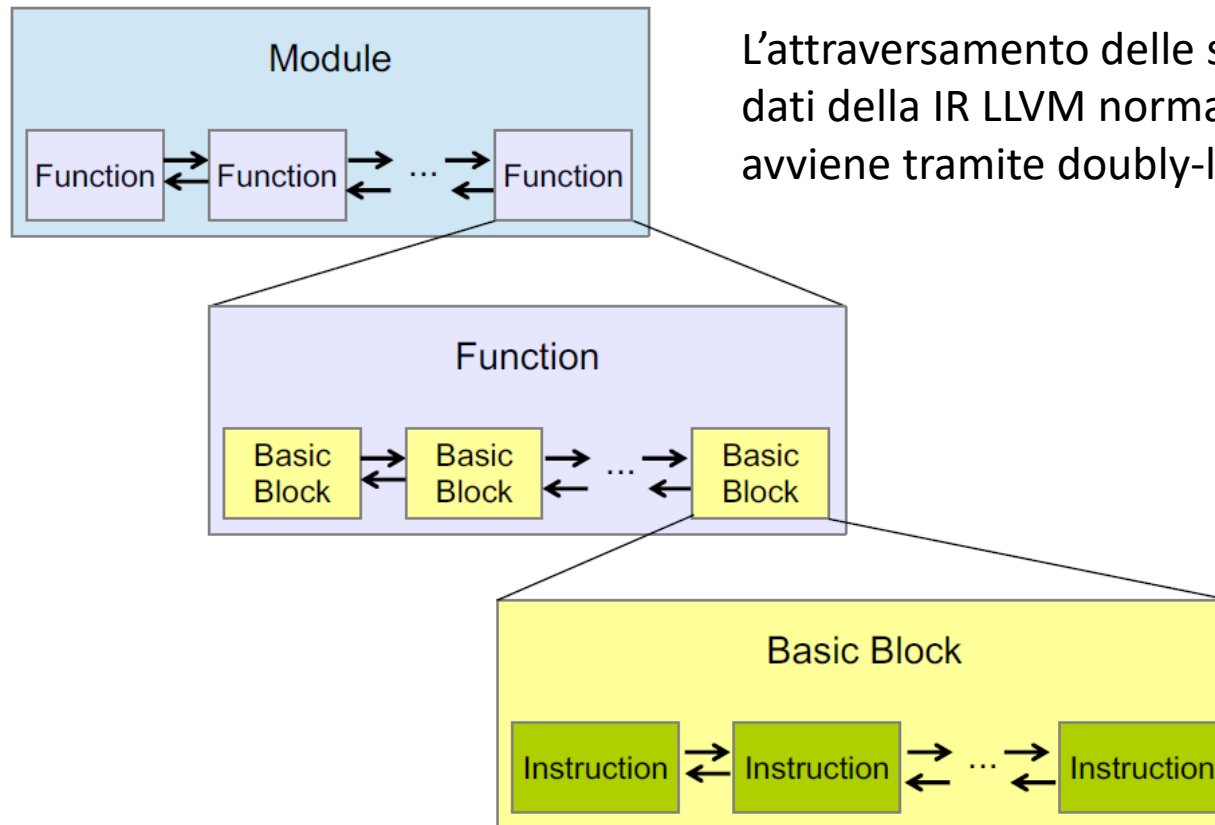
Il nostro programma

- Files 
- Funzioni 
- Basic Blocks 
- Istruzioni 

Un Modulo LLVM

- Module
 - lista di Function e variabili globali
- Function
 - lista di BasicBlocks e argomenti
- BasicBlock
 - lista di Instruction
- Instruction
 - Opcode e operandi

Iteratori



L'attraversamento delle strutture dati della IR LLVM normalmente avviene tramite doubly-linked lists

Iteratori

```
Module &M = ...;  
for (auto iter = M.begin(); iter != M.end(); ++iter) {  
    Function &F = *iter;  
    // do some stuff for each function  
}
```

- **Nota:**
 - Sintassi simile a quella del container STL `vector`.

Downcasting

- Perché ci serve il **Downcasting**?
- Immaginiamo di avere una `Instruction`
 - Come facciamo a sapere se è un'istruzione unaria o binaria?
 - O capire se è una branch instruction o una call instruction?
- Il **Downcasting** ci aiuta a recuperare maggiore informazione dagli `Iterators`
- Es.:

```
Instruction *inst = ...  
if (CallInst *call_inst = dyn_cast<CallInst>(inst))  
    outs() << "I am a call instruction"  
        << std::endl;
```

Downcasting

```
// Terminator Instructions
FIRST_TERM_INST ( 1)
HANDLE_TERM_INST ( 1, Ret, ReturnInst)
HANDLE_TERM_INST ( 2, Br, BranchInst)
HANDLE_TERM_INST ( 3, Switch, SwitchInst)
...
```

- Perché ci serve il **Downcasting**?
- Immaginiamo di avere una `Instruction`
 - Come facciamo a sapere se è un'istruzione unaria o binaria?
 - O capire se è una branch instruction o una call instruction?
- Il **Downcasting** ci aiuta a recuperare maggiore informazione dagli `Iterators`
- Es.:

Tutte le sottoclassi di `Instruction` sono definite in
`#include "llvm/IR/Instruction.def"`

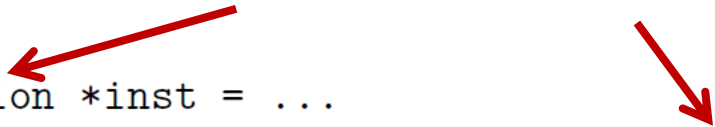
```
Instruction *inst = ...
if (CallInst *call_inst = dyn_cast<CallInst>(inst))
    outs() << "I am a call instruction"
    << std::endl;
```

Per eseguire questo codice ha bisogno di `#include "llvm/IR/Instructions.h"`

Downcasting

- Perché ci serve il **Downcasting**?
- Immaginiamo di avere una `Instruction`
 - Come facciamo a sapere se è un'istruzione unaria o binaria?
 - O capire se è una branch instruction o una call instruction?
- Il **Downcasting** ci aiuta a recuperare maggiore informazione dagli `Iterators`
- Es.:

N.B. La classe `Instruction` è solo un esempio. Si può effettuare il downcasting sulle classi più disparate



```
Instruction *inst = ...  
if (CallInst *call_inst = dyn_cast<CallInst>(inst))  
    outs() << "I am a call instruction"  
    << std::endl;
```

Interfacce dei passi LLVM (vecchio pass manager)

- Fino alla versione 14 LLVM forniva diverse interfacce per i passi
 - `BasicBlockPass`: itera sui *basic blocks*
 - `CallGraphSCCPass`: itera sui nodi del *call graph*
 - `FunctionPass`: itera sulla lista delle funzioni nel modulo
 - `LoopPass`: itera sui *loops*, in ordine inverso di nesting
 - `ModulePass`: generico passo interprocedurale
 - `RegionPass`: itera sulle SESE *regions*, in ordine inverso di nesting

```
struct MyPass : public FunctionPass
```

- Modello basato su ereditarietà
 - Infrastruttura rigida
 - Difficile comporre pipeline
 - Scarsa separazione tra analisi e trasformazioni

Interfacce dei passi LLVM (vecchio pass manager)

- Fino alla versione 14 LLVM forniva diverse interfacce per i passi
 - `BasicBlockPass`: itera sui *basic blocks*
 - `CallGraphSCCPass`: itera sui nodi del *call graph*
 - `FunctionPass`: itera sulla lista delle funzioni nel modulo
 - `LoopPass`: itera sui *loops*, in ordine inverso di nesting
 - `ModulePass`: generico passo interprocedurale
 - `RegionPass`: itera sulle SESE *regions*, in ordine inverso di nesting

```
struct MyPass : public FunctionPass
```

- Modello basato su ereditarietà
 - Infrastruttura rigida
 - Difficile comporre pipeline
 - Scarsa separazione tra analisi e trasformazioni

MyPass eredita da **FunctionPass**. Opera a livello di **Function**

Interfacce dei passi LLVM (New Pass Manager)

- Nel nuovo pass manager si adotta un modello parametrico basato su template.

```
struct MyPass : PassInfoMixin<MyPass>

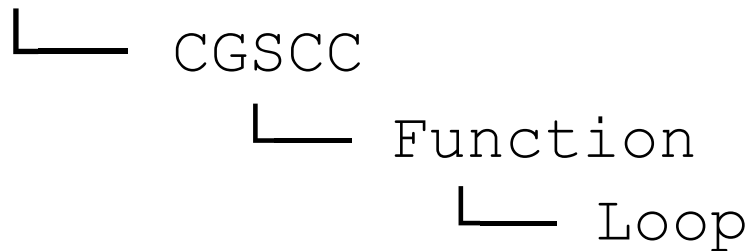
// Module
run(Module&, ModuleAnalysisManager&)
// Function
run(Function&, FunctionAnalysisManager&)
// Loop
run(Loop&, LoopAnalysisManager&, ...)
...
```

- Modello molto più pulito rispetto al passato

Interfacce dei passi LLVM (New Pass Manager)

- Nel nuovo pass manager si adotta un modello parametrico basato su template.
- I livelli sono solo quattro, gerarchicamente innestati

Module



Interfacce dei passi LLVM (New Pass Manager)

- Nel nuovo pass manager si adotta un modello parametrico basato su template.
- I livelli sono solo quattro, gerarchicamente innestati

PASSO CHE OPERA SU IR DI TIPO:

Module

└─ CGSCC

└─ Function

└─ Loop

Module

Call graph Strongly Connected Component

Function

Loop

Pass manager e Pass Builder

- Il pass manager è il gestore dei passi di ottimizzazione applicati alla IR sorgente
<https://llvm.org/docs/NewPassManager.html>
- Nelle nuove versioni di LLVM ce ne sono quattro; uno per ogni livello
- Il PassBuilder costruisce pipeline standard (-O2, -O3, ecc.)
 - Sequenza predefinita (ma flessibile) di passi
- Permette estensione delle pipeline standard tramite plugin
 - L'opzione che useremo noi
- Il default si può alterare invocando una sequenza arbitraria di passi tramite linea di comando

```
$ opt -passes='pass1,pass2' /tmp/a.ll -S  
# -p is an alias for -passes  
$ opt -p pass1,pass2 /tmp/a.ll -S
```

<https://llvm.org/docs/CommandGuide/opt.html>

Analysis manager

- Sappiamo che i passi possono anche essere di sola analisi della IR
- Nel vecchio Pass Manager le analisi erano “pass speciali”
 - caching complicato
 - invalidazione poco chiara
- Nel Nuovo Pass Manager ogni livello ha il proprio analysis manager:
 - ModuleAnalysisManager
 - CGSCCAnalysisManager
 - FunctionAnalysisManager
 - LoopAnalysisManager

Analysis manager

- **Gli Analysis manager gestiscono:**

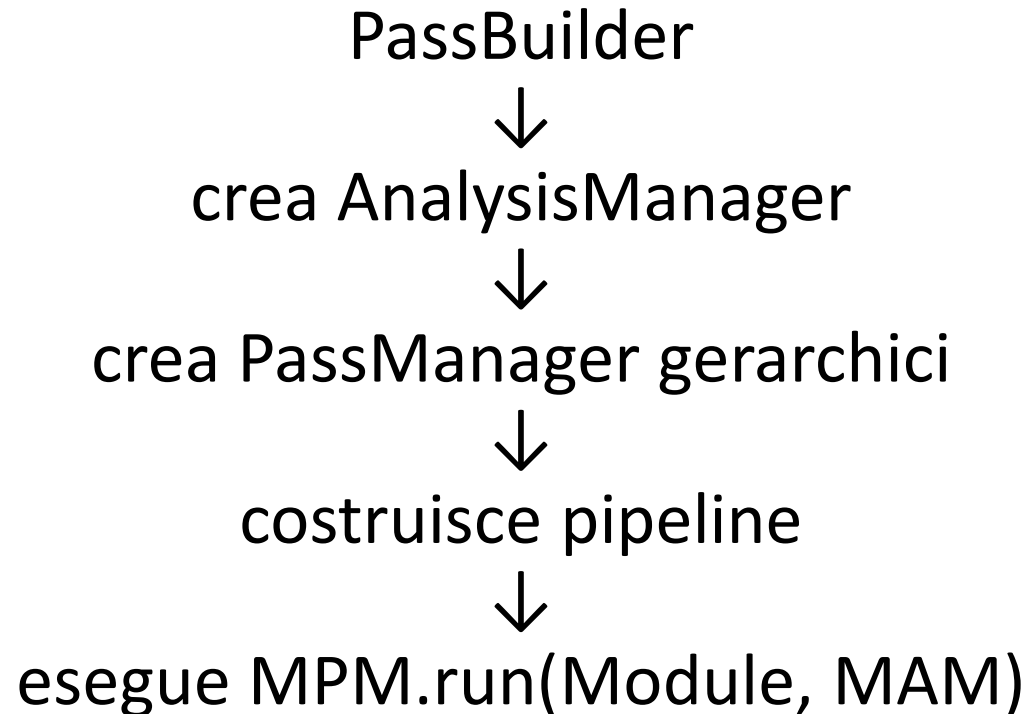
- Calcolo lazy delle analisi
- Caching
- Invalidazione
- Dipendenze tra analisi

- **Dentro un Function pass:**

```
auto &DT = AM.getResult<DominatorTreeAnalysis>(F);
```

- Se non esiste → viene calcolata
- Se esiste → viene riusata
- Se è invalida → viene ricalcolata

Struttura completa





UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

LAB 1 – TEST PASS

Test Pass – Sorgente

- Scaricate dalla pagina Moodle i files di test per l'esercitazione 1
 - `Fibonacci.c`
 - `Loop.c`

Esercizio 1 – IR e CFG

- Per produrre la IR da dare in pasto al middle-end ci serve **clang** (il *frontend*)
 - produce "assembly" IR
 - solo compilazione (genera assembly)

```
clang -O2 -emit-llvm -S -c test/Loop.c \
-o test/Loop.ll
```

output file

input file

- `-S` → ci fermiamo prima dell'assemblaggio
 - produciamo assembly (.s) human readable anziché un file oggetto (.o)
- `-c` → ci fermiamo prima del linking
 - produciamo un file oggetto (.o, non linkato)
 - Non necessario se specifichiamo `-S` (ci fermiamo comunque prima del linking)
- `-emit-llvm` → L'assembly prodotto è quello "virtuale" della IR

Esercizio 1 – IR e CFG

- Oppure si può prima produrre il *bytecode* e poi disassemblare per produrre la forma assembly

produce "object code" IR

solo object code (no linking)

- `clang -O2 -emit-llvm -c test/Loop.c \`
`-o test/Loop.bc`
- `llvm-dis test/Loop.bc -o=./test/Loop.ll`

Esercizio 1 – IR e CFG

- Per ognuno dei test benchmarks produrre la IR con clang e analizzarla, cercando di capire cosa significa ogni parte
- Disegnare il CFG per ogni funzione

Esercizio 1 – IR e CFG

```
int g;  
  
int g_incr(int c) {  
    g += c;  
    return g;  
}
```

SORGENTE

```
@g = dso_local local_unnamed_addr global i32 0, align 4  
  
define dso_local i32 @g_incr(i32 @noundef %0) local_unnamed_addr #0  
{  
    %2 = load i32, i32* @g, align 4, !tbaa !3  
    %3 = add nsw i32 %2, %0  
    store i32 %3, i32* @g, align 4, !tbaa !3  
    ret i32 %3  
}
```

LLVM IR

Esercizio 1 – IR e CFG

```
int g;  
  
int g_incr(int c) {  
    g += c;  
    return g;  
}
```

symbol will resolve within
the same local unit

SORGENTE

The address of the object is not
known within the module

```
@g = dso_local local_unnamed_addr global i32 0, align 4  
  
define dso_local i32 @g_incr(i32 noundef %0) local_unnamed_addr #0  
{  
    %2 = load i32, i32* @g, align 4, !tbaa !3  
    %3 = add nsw i32 %2, %0  
    store i32 %3, i32* @g, align 4, !tbaa !3  
    ret i32 %3  
}
```

Function definition

Attributes are described later in the file

LLVM IR

Esercizio 1 – IR e CFG

```
; Function Attrs: nofree norecurse nosync nounwind uwtable
define dso_local i32 @loop(i32 noundef %0, i32 noundef %1, i32
noundef %2) local_unnamed_addr #1 {
    %4 = load i32, i32* @g, align 4, !tbaa !3
    %5 = icmp sgt i32 %1, %0
    br i1 %5, label %6, label %10

6:                                ; preds = %3
    %7 = sub i32 %1, %0
    %8 = mul i32 %7, %2
    %9 = add i32 %4, %8
    store i32 %9, i32* @g, align 4, !tbaa !3
    br label %10

10:                               ; preds = %6, %3
    %11 = phi i32 [ %9, %6 ], [ %4, %3 ]
    ret i32 %11 h
}
```

LLVM IR

```
int loop(int a, int b, int c) {
    int i, ret = 0;
    for (i = a; i < b; i++)
        g_incr(c);
}
```

SORGENTE

Esercizio 1 – IR e CFG

```
; Function Attrs: nofree norecurse nosync nounwind uwtable
define dso_local i32 @loop(i32 noundef %0, i32 noundef %1, i32
noundef %2) local unnamed addr #1 {
```

```
    %4 = load i32, i32* @g, align 4, !tbaa !3
    %5 = icmp sgt i32 %1, %0
    br i1 %5, label %6, label %10
```

BB1

LLVM IR

```
6:                                ; preds = %3
    %7 = sub i32 %1, %0
    %8 = mul i32 %7, %2
    %9 = add i32 %4, %8
    store i32 %9, i32* @g, align 4, !tbaa !3
    br label %10
```

BB2

```
10:                               ; preds = %6, %3
    %11 = phi i32 [ %9, %6 ], [ %4, %3 ]
    ret i32 %11
}
```

BB3

```
int loop(int a, int b, int c) {
    int i, ret = 0;
    for (i = a; i < b; i++)
        g_incr(c);
}
```

SORGENTE

Esercizio 1 – IR e CFG

```
; Function Attrs: nofree norecurse nosync nounwind uwtable
define dso_local i32 @loop(i32 noundef %0, i32 noundef %1, i32
noundef %2) local_unnamed_addr #1 {
    %4 = load i32, i32* @g, align 4, !tbaa !3
    %5 = icmp sgt i32 %1, %0
    br i1 %5, label %6, label %10

6:                                     ; preds = %3
    %7 = sub i32 %1, %0
    %8 = mul i32 %7, %2
    %9 = add i32 %4, %8
    store i32 %9, i32* @g, align 4, !tbaa !3
    br label %10

10:                                   ; preds = %6, %3
    %11 = phi i32 [ %9, %6 ], [ %4, %3 ]
    ret i32 %11
}
```

LLVM IR

```
int loop(int a, int b, int c) {
    int i, ret = 0;
    for (i = a; i < b; i++)
        g_incr(c);
}
```

SORGENTE

Cos'è successo al loop?

Esercizio 1 – IR e CFG

- Perché il codice è cambiato così tanto rispetto al programma C originale?
 - Provate ad aggiungere il flag `-Rpass=.*` al comando di invocazione di *clang* (con `-O2`)

Vi viene stampato come tutti i passi (.*) di ottimizzazione trasformano il codice

Esercizio 1 – IR e CFG

- Provare ora a rigenerare la IR con l'opzione `-O0` al comando di invocazione di *clang*
- Questo disabilita completamente la pipeline di ottimizzazione, *il codice è solo tradotto da C a IR*
- *Ma c'è un nuovo problema...*

Esercizio 1 – IR e CFG

- ***Sono comparse tantissime load/store***

```
; Function Attrs: noline nounwind optnone uwtable
define dso_local i32 @g_incr(i32 noundef %0) #0 {
    %2 = alloca i32, align 4
    store i32 %0, ptr %2, align 4
    %3 = load i32, ptr %2, align 4
    %4 = load i32, ptr @g, align 4
    %5 = add nsw i32 %4, %3
    store i32 %5, ptr @g, align 4
    %6 = load i32, ptr @g, align 4
    ret i32 %6
}
```

- Di base la IR assume di non avere nessun registro, quindi tutte le operazioni sui dati sfruttano *load/store*
- Il passo `mem2reg` viene eseguito nelle fase iniziali, per trasformare tutte le *load/store* che non sono veri accessi in memoria in letture/scritture su registri virtuali.
- Con `-O0` viene disattivato anche il passo `mem2reg`

Esercizio 1 – IR e CFG

- Quindi prima di analizzare la IR prodotta con `-O0` dobbiamo applicare manualmente `mem2reg`
- `opt -passes=mem2reg test/Loop.ll \`
`-S -o test/Loop.m2r.ll`
- Cos'è successo alla IR?
 - Purtroppo, niente!!!

Esercizio 1 – IR e CFG

- `clang -O0 -emit-llvm -S -c test/Loop.c \`
`-o test/Loop.ll`

- Applicare `-O0` al comando `clang` ha l'effetto di marcare tutte le funzioni nell'intermedio con l'attributo `optnone`, che rende inefficaci tutte le successive ottimizzazioni applicate

- Si può vedere nella IR nella sezione `attributes`



```
; Function Attrs: noinline nounwind optnone uwtable
```

- Come possiamo evitare questo problema?

- Aggiungiamo a clang i flag `-Xclang -disable-O0-optnone`

Esercizio 1 – IR e CFG

- `clang -O0 -Xclang -disable-O0-optnone \`
`-emit-llvm -S -c test/Loop.c \`
`-o test/Loop.O0.ll`
- `opt -passes=mem2reg test/Loop.O0.ll \`
`-S -o test/Loop.O0.m2r.ll`
- Riuscite a riconoscere il funzionamento del programma originale?
- Confrontate `Loop.O2.ll` **VS** `Loop.O0.m2r.ll`

Esercizio 2 - TestPass

<https://llvm.org/docs/WritingAnLLVMNewPMPass.html>

- Per questo corso scriveremo i nostri passi di analisi e ottimizzazione come plugin per il pass manager di LLVM
- Questo ci consente uno sviluppo esterno al build tree di LLVM
 - Il compilato verrà poi linkato come una dynamic library
- Createvi un workspace con una directory root
- Es.
 - `mkdir Laboratori_Compilatori`
 - `export ROOT_LABS=<path/to>Laboratori_Compilatori`

Esercizio 2 - TestPass

<https://llvm.org/docs/WritingAnLLVMNewPMPass.html>

- Scaricate da Moodle in ROOT_LABS/Lab1 i due file
 - TestPass.cpp
 - CMakeLists.txt
- Quindi provate a settare l'environment e compilare
 - export
LLVM_DIR=<installation/dir/of/llvm/19>/bin
 - mkdir build
 - cd build
 - cmake -DLLVM_INSTALL_DIR=\$LLVM_DIR
<source/dir/test/pass>/
 - make

**L'output di questa procedura è la generazione
della libreria dinamica libTestPass.so**

Andrea Marongiu - Compilatori - 2025/2026

Esercizio 2 - TestPass

- Possiamo ora invocare l'ottimizzatore (opt) con il flag per fare overriding del pass manager di default
 - indichiamo noi la lista di passi da applicare

- `cd ..` *// torniamo alla cartella Lab1*

- `opt -load-pass-plugin`
 `<path/to/build/dir>/libTestPass.so` \
- `-passes=test-pass test/Loop.bc` \
- `-o test/LoopTestPass.bc`

oppure -p test-pass

oppure 'll' a seconda del formato col quale si sta lavorando

Esercizio 2 - TestPass

- Possiamo ora invocare l'ottimizzatore (opt) con il flag per fare overriding del pass manager di default
 - indichiamo noi la lista di passi da applicare

- `opt -passes=test-pass test/Loop.bc \`
`-o test/LoopTestPass.bc`



dal momento che il nostro è un passo di analisi che non produce output (non altera la IR) possiamo disabilitare l'output

Rimpiazzare il flag `-o ...` con `-disable-output`

Esercizio 2 - TestPass

- Se avete fatto tutto correttamente l'esecuzione del passo su `Loop.ll` dovrebbe produrre questo output:

```
g_incr  
loop
```

- Qui comincia il nostro lavoro...

Esercizio 2 - TestPass

- Estendete il passo `TestPass` di modo che analizzi la IR e stampi alcune informazioni utili per ciascuna delle funzioni che compaiono nel programma di test
 1. Nome
 2. Numero di argomenti ('N+*' in caso di funzione variadica)(*)
 3. Numero di chiamate a funzione nello stesso modulo
 4. Numero di *Basic Blocks*
 5. Numero di *Istruzioni*
- (*) es., per la funzione

```
int printf (const char *format, ...);
```

bisogna stampare '1+*'

Esercizio 2 - TestPass

- La documentazione è nostra amica
 - Indice delle classi per la IR LLVM
 - <https://llvm.org/doxygen/classes.html>